

s3.01 — Asynchronous control-plane resource API — create / update / delete

Index

- Interview framing
- Requirements — IC3 baseline
- Capacity and consequence — IC3 baseline
- Architecture and flow — IC3 baseline
- API and data model — IC3 baseline
- Acceptance and durable state transitions — IC3 baseline
- External effects and cancellation — IC3 baseline
- Failure, operations, and security
- Alternatives and IC4 stretch
- Level signal — IC3 baseline vs IC4 stretch
- Interview pushback and validation
- Recall card
- References

Interview framing

- Target: OCI IC3 ownership of a bounded platform component.
 - IC4 stretch changes the architecture only when a stronger requirement earns the cost.
 - These are preparation labels, not Oracle's official rubric.
- Evidence: [ranked bank](#).
 - Recruiter control-plane focus and OCI work-request documentation support this preparation topic.
 - This is a proposed design, not a claim about OCI internals or a reported accepted answer.
- Why this is hard
 - Naive: keep the HTTP request open while a worker provisions the resource.
 - First break: the provider completes after the client times out, and a retry provisions a second resource.
 - Second break: a worker whose lease expired publishes a stale terminal state over a newer attempt, or a status replica reports SUCCEEDED and then IN_PROGRESS.
 - Minimum primitives
 - Durable operation identity.
 - Resource generation and conditional transitions.
 - Transactional outbox.
 - Reconciliation of uncertain external effects.
- 🗣️ “A 202 acknowledges durable intent; only reconciliation can resolve an uncertain provider effect.”
- Opening script
 - “I will separate accepting a resource operation from completing its external work.”
 - “I will first confirm the consistency promised by status reads and whether provider calls support idempotency.”
 - “The API will commit the resource, work request, and outbox event together before returning 202.”
 - “The deep dive is recovery after a worker loses the provider response.”
 - “I will close with tenant fairness, cancellation limits, and the cost of a stronger cross-region contract.”

Phase	Clock	Deliverable
Intro	00:00–05:00	Establish role context and confirm the prompt
Requirements	05:00–11:00	Scope and one correctness invariant

Phase	Clock	Deliverable
Capacity and API	11:00–17:00	A number that forces a choice; retry contract
Architecture	17:00–30:00	Draw ownership and trace one request
Deep dive	30:00–45:00	Concurrency proof and a crash boundary
Operations and alternatives	45:00–50:00	Degradation, recovery, and flip condition
Wrap-up	50:00–54:00	Four-sentence recap and unresolved assumption
Interviewer questions	54:00–60:00	Team ownership and production priorities

- Clock note
 - The design slice is 45 minutes within the 60-minute round.
 - If the interviewer gives 45 minutes total, use a 2-minute intro and compress the remaining phases.
 - The interviewer's clock wins.

Requirements — IC3 baseline

- Numerical targets are interview assumptions unless an adjacent primary source states otherwise.

Question	Assumed answer	Why it changes the design
How long does provisioning take?	Mean 120 s; tail 20 min	Use durable asynchronous state
Can providers deduplicate creates?	Some accept a stable operation token	Reconcile the others instead of promising exactly-once effects
Concurrent update/delete?	One accepted mutation per resource generation	Use If-Match and a resource state machine
Read consistency?	Authoritative status by operation ID	Avoid apparent success-to-running regressions
Cancellation?	Best effort before an irreversible step	Expose compensation separately

- Functional scope
 - Create resources asynchronously.
 - Update and delete with expected resource versions.
 - Poll work requests and retrieve structured failures.
 - Retry an accepted request without creating a new logical operation.
 - Reconcile uncertain effects and abandoned work.
- Targets below are interview assumptions, not product SLAs.

Requirement	Target	Enforced by
Acceptance latency	p99 < 200 ms	One short metadata transaction
Lifecycle API availability	99.9% regional target	Replicated store and stateless API
Lost-response retry horizon	24 h	Durable tenant/operation/token record
Status visibility	p99 < 2 s after commit	Authoritative reads; bounded notification lag
Recovery detection	< 60 s after worker lease expiry	Indexed lease reaper
Tenant isolation	100 active operations per tenant by default	Durable admission permits

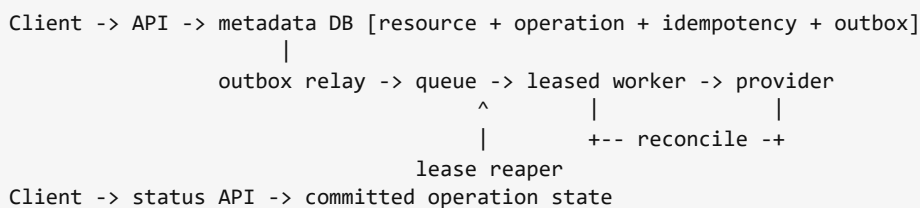
- Invariants
 - One idempotency identity selects one canonical request hash and work-request ID.
 - Only the current resource generation and worker fence may publish a transition.
 - SUCCEEDED means the provider outcome was verified and resource metadata committed.
 - A queue acknowledgment never substitutes for the metadata commit.
- Non-goals
 - Atomic transactions across arbitrary cloud providers.
 - Instant cancellation of completed physical effects.

- Every OCI API exposing identical work-request behavior.

Capacity and consequence — IC3 baseline

- All workload values below are sizing assumptions.
- Accepted operations
 - $500/s \times 120 s = 60,000$ active operations by Little's law.
 - Decision: workers wait asynchronously for provider completion instead of holding 60,000 threads.
- Polling
 - $60,000 / 5 s = 12,000$ status reads/s before backoff.
 - Decision: use ETags and jittered polling with longer intervals for long-running steps.
- History
 - $500/s \times 2 KB \times 86,400 = 86.4 GB/day$ of operation records.
 - 90 days = 7.776 TB before indexes and replication.
 - Decision: retain a compact searchable hot record and tier detailed history.
- Recovery burst
 - A 10-minute queue outage accumulates 300,000 operations at this rate.
 - Decision: cap catch-up concurrency against provider quotas rather than draining at maximum worker speed.

Architecture and flow — IC3 baseline



- API
 - Buys durable acceptance with one transaction.
 - Pays a metadata quorum round trip.
 - Queue and relay
 - Buy buffering and retryable delivery.
 - Pay duplicate delivery and backlog management.
 - Worker
 - Owns an attempt lease and persists provider correlation IDs.
 - Pays reconciliation complexity when the provider outcome is unknown.
 - Metadata store
 - Owns resource versions and terminal transitions.
 - Pays serialized writes for conflicting operations on one resource.
- Authenticate tenant and authorize the exact operation.
- Resolve the idempotency token and compare the canonical payload hash.
- Commit resource intent, operation, and outbox in one transaction.
- Return 202 with operation ID and polling URL.
- Claim an attempt fence and execute or reconcile the provider operation.
- Commit the observed result and completion outbox under expected state and fence.

API and data model — IC3 baseline

- Proposed API contracts.

- Authenticate the tenant from credentials before looking up any resource.
- Retain idempotency records for the documented retry horizon.

Surface	Contract	Idempotency and conflicts
POST /resources	202 plus resourceId and workRequestId	Same tenant/action/token and hash returns the same IDs; another hash is 409
PATCH /resources/{id}	202; If-Match required	A stale resource version is 412; a conflicting active mutation is 409
DELETE /resources/{id}	202 until verified deletion	Repeated semantic delete returns its operation; already absent may return 204
GET /work-requests/{id}	State, version, timestamps, errors; ETag	Authoritative read; optional minVersion waits or returns retryable unavailable
POST /work-requests/{id}:cancel	202 cancellation requested	A completed effect is not undone; terminal state returns the existing result

State	Key	Fields and invariant
Resource	(tenant, resourceId)	Generation, desired state, observed state, active operation
Operation	(tenant, operationId)	State, transitionVersion, providerId, attemptFence, leaseUntil
Idempotency	(tenant, action, token)	Canonical hash and operation ID; unique constraint
Outbox	(operationId, transitionVersion)	Payload and delivery status committed with state
Step intent	(operationId, stepId)	Stable provider token and observed outcome

- Reference DDL
 - Keys and uniqueness match the state table above and the [companion schema](#).
 - The queue holds only copies of outbox rows
 - The metadata database is the single state authority.
 - Provider-side objects are never mirrored beyond provider_id and provider_ref.

```

CREATE TABLE resources (
  tenant_id text NOT NULL, resource_id text NOT NULL,
  kind text NOT NULL, name text NOT NULL,
  generation bigint NOT NULL DEFAULT 1,          -- bumps on every accepted mutation; source of the ETag
  desired_spec jsonb NOT NULL,
  observed_state jsonb NOT NULL DEFAULT '{}', observed_generation bigint NOT NULL DEFAULT 0,
  lifecycle_state text NOT NULL,
  active_operation_id text,                      -- at most one accepted mutation per generation; no FK
  to avoid a cycle
  created_at timestamptz NOT NULL, deleted_at timestamptz, -- tombstone until downstream cleanup is
  verified
  PRIMARY KEY (tenant_id, resource_id),
  UNIQUE (tenant_id, kind, name),
  CHECK (observed_generation <= generation),
  CHECK (lifecycle_state IN ('CREATING', 'ACTIVE', 'UPDATING', 'DELETING', 'DELETED', 'FAILED'))
);
CREATE TABLE operations (
  tenant_id text NOT NULL, operation_id text NOT NULL, resource_id text NOT NULL,
  action text NOT NULL, target_generation bigint NOT NULL,
  state text NOT NULL,
  transition_version bigint NOT NULL DEFAULT 0,    -- monotonic; every committed transition increments it
  provider_id text,                               -- provider operation ID as soon as it is known
  attempt_fence bigint NOT NULL DEFAULT 0, lease_until timestamptz,
  cancel_requested_version bigint,                -- transition_version at which cancellation was recorded
  accepted_at timestamptz NOT NULL, finished_at timestamptz, errors jsonb,
  PRIMARY KEY (tenant_id, operation_id),
  FOREIGN KEY (tenant_id, resource_id) REFERENCES resources,
  CHECK (state IN ('ACCEPTED', 'IN_PROGRESS', 'RECONCILING', 'CLEANUP_REQUIRED',
    'SUCCEEDED', 'FAILED', 'CANCELED')),
  CHECK (attempt_fence >= 0 AND transition_version >= 0)
);
CREATE INDEX operations_by_resource ON operations
  (tenant_id, resource_id, accepted_at DESC, operation_id);
CREATE INDEX operations_lease_recovery ON operations (lease_until)
  WHERE state IN ('IN_PROGRESS', 'RECONCILING');
CREATE TABLE idempotency_records (
  tenant_id text NOT NULL, action text NOT NULL, token text NOT NULL,
  request_hash text NOT NULL,                    -- canonical payload hash; mismatch is 409
  operation_id text NOT NULL, resource_id text NOT NULL,
  created_at timestamptz NOT NULL, expires_at timestamptz NOT NULL, -- created_at + 24 h retry horizon
  PRIMARY KEY (tenant_id, action, token),
  FOREIGN KEY (tenant_id, operation_id) REFERENCES operations DEFERRABLE INITIALLY DEFERRED,
  CHECK (expires_at > created_at)
);
CREATE INDEX idempotency_expiry ON idempotency_records (expires_at);
CREATE TABLE outbox (
  tenant_id text NOT NULL, operation_id text NOT NULL, transition_version bigint NOT NULL,
  event_type text NOT NULL, payload jsonb NOT NULL,
  created_at timestamptz NOT NULL, published_at timestamptz,
  PRIMARY KEY (tenant_id, operation_id, transition_version), -- relay replays by this key
  FOREIGN KEY (tenant_id, operation_id) REFERENCES operations
);
CREATE INDEX outbox_unpublished ON outbox (created_at) WHERE published_at IS NULL;
CREATE TABLE step_intents (
  tenant_id text NOT NULL, operation_id text NOT NULL, step_id text NOT NULL,
  provider_token text NOT NULL,                  -- stable token sent to the provider; reused on every
  retry
  attempt_fence bigint NOT NULL,
  provider_ref text,                             -- provider object or operation ID once observed
  outcome text,                                  -- NULL until observed; UNKNOWN forces RECONCILING
  created_at timestamptz NOT NULL, observed_at timestamptz,
  PRIMARY KEY (tenant_id, operation_id, step_id),
  FOREIGN KEY (tenant_id, operation_id) REFERENCES operations,
  CHECK (outcome IS NULL OR outcome IN ('SUCCEEDED', 'FAILED', 'UNKNOWN'))
);

```

- What the DDL does not capture

- Queue messages are copies of `outbox` rows; the relay's delivery cursor and the worker's in-flight step are not relational state.
- Provider-side resources exist only behind `provider_id`, `provider_token`, and `provider_ref`; a local rollback never undoes them.
- Deviation from the companion
 - Column names are the snake_case form of the state table: `transitionVersion` → `transition_version`, `attemptFence` → `attempt_fence`, `leaseUntil` → `lease_until`, `providerId` → `provider_id`, `stepId` → `step_id`.
 - `attempt_fence` replaces the companion's `worker_fence`; the companion's `operation_events` table is replaced by `outbox` and `step_intents`, which the state table names.
 - `idempotency_records` is this file's own table keyed (`tenant_id`, `action`, `token`) instead of the shared `api_intents`; `outbox` and `step_intents` gain a `tenant_id` prefix.
 - `active_operation_id` and `cancel_requested_version` are additions the deep dive relies on.

Acceptance and durable state transitions — IC3 baseline

- Acceptance transaction
 - Insert the idempotency row using a conflict-aware insert.
 - On a matching existing token, return its operation after checking the hash.
 - On a new token, lock the resource and validate its expected generation.
 - Insert the operation and outbox before committing.
 - A rolled-back transaction exposes no accepted operation.
- Transition contract
 - `ACCEPTED` → `IN_PROGRESS` → `SUCCEEDED` is the successful path.
 - Failure may enter `RECONCILING` or `CLEANUP_REQUIRED` before a terminal outcome.
 - Persist each transition with a monotonically increasing version.
 - A stale worker must fail the expected-state and attempt-fence check.

```
CLAIM(operation):
  transaction:
    lock operation
    require runnable state and no unexpired attempt lease
    increment attemptFence; persist leaseUntil
    return immutable step intent and attemptFence
COMPLETE(operation, fence, expectedVersion, verifiedResult):
  transaction:
    require current fence and unexpired lease
    require transitionVersion == expectedVersion and state permits completion
    require resource.activeOperation == operation
    update operation and resource atomically
    insert outbox(operation, next transitionVersion)
```

- Status reads
 - Read the committed authority for the baseline contract.
 - A cached or replica response must meet a requested minimum version or decline the read.
 - Percentage completion is advisory per attempt.
 - Report 100% only with committed success.

External effects and cancellation — IC3 baseline

- Provider supports idempotency
 - Persist a stable step token before calling the provider.
 - Retry with that token within its documented retention window.
 - Store the provider operation ID as soon as it becomes known.
- Provider does not support idempotency

- Attach a searchable operation tag when possible.
- After a timeout, query by correlation identity before retrying creation.
- If absence cannot be established safely, expose RECONCILING and require bounded manual resolution.
- A local SQL transaction cannot guarantee one physical provider effect.
- Cancellation
 - Record the cancellation request under the current operation version.
 - Workers check it before each irreversible step.
 - A completed allocation needs a compensating delete with its own intent and retry policy.
 - Cancellation may lose a race with completion.
- Notifications
 - Publish completion through the transactional outbox.
 - Deduplicate events by operation ID and transition version.
 - Sign webhook bodies with an audience and timestamp.
 - Validate callback destinations against SSRF and enforce a replay window.
 - Notifications accelerate discovery
 - Polling remains the recovery path.

Failure, operations, and security

Failure	Blast radius	Detection	Degraded behavior	Recovery
API reply lost	One client operation	Token lookup	Retry returns the same accepted operation	Replay stored identity
Worker dies after provider create	One resource; possible orphan	Expired lease and unresolved step	RECONCILING; no blind duplicate create	Query provider by stable identity
Queue unavailable	New execution delayed	Oldest outbox and queue age	Durable 202 still possible within backlog admission limit	Replay outbox at quota-limited rate
Provider outage	Operations using that provider	Latency and error budget	Backoff; retain durable intent	Resume with original step identity
Metadata quorum lost	Regional mutations	Commit/read failures	503; do not claim acceptance	Restore quorum and reconcile leases
Stale worker publishes late	One operation	Fence or transitionVersion mismatch at commit	Late write rejected; current attempt unaffected	No action; the current attempt owns the outcome
Outbox relay stalls	All new executions in the region	Oldest undelivered outbox row age	Acceptance continues until the backlog admission limit	Restart the relay; replay by (operationId, transitionVersion)
Idempotency record expired before a client retry	One client operation; possible duplicate logical operation	Retry arrives with an unknown token after 24 h	Treated as a new request	Document the horizon; clients poll by operation ID after it
Notification endpoint down	One tenant's callbacks	Webhook delivery failures	Polling remains authoritative	Retry with backoff; stop after the deadline and keep the operation state

- Observe
 - Acceptance p99.
 - Oldest nonterminal operation age.
 - Provider-unknown outcomes.
 - Reconciliation backlog by tenant.
- Retry discipline
 - Use exponential backoff with jitter and an end-to-end operation deadline.
 - Separate transient provider errors from invalid requests.

- Respect provider `Retry-After` and quota hints so catch-up does not trip the provider's throttle.
- A validation or authorization failure needs a corrective action, not another attempt.
- Tenant isolation
 - Enforce durable admission permits and provider-specific concurrency limits.
 - Bind operation lookup and provider credentials to the authenticated tenant.
 - Scope idempotency records to `(tenant, action, token)` so one tenant cannot replay another tenant's operation.
 - A tenant's token receives `404` on another tenant's resource or operation IDs, not `403`, to avoid existence leaks.
 - Drain post-outage catch-up per tenant under the same permits, so one tenant's backlog cannot starve the others.
- Rollout and rollback safety
 - Use expand/contract state changes and readers that tolerate unknown enum values.
 - Canary a worker version on one resource type.
 - Keep old workers from claiming step formats they cannot interpret.
 - Canary the API fleet by fault domain so a bad deploy pauses acceptance in one domain, not the region.
 - Static stability: already provisioned resources keep serving while this control plane deploys or fails, because the data plane never depends on it.
 - Rolling back a worker cannot un-publish a transition because `transitionVersion` only increases.
 - A new worker must claim step intents persisted by the previous version, or the rollout must drain in-flight attempts first.
 - Contract the schema only after no old reader remains, so an API rollback never meets a column it cannot read.

Alternatives and IC4 stretch

Option	What it is	Strength	Weakness	Choose when
SQL plus outbox	One transactional state authority	Simple atomic acceptance	Metadata write bottleneck	Baseline bounded regional service
Workflow engine	Durable workflow execution	Timers and recovery machinery	Operational dependency and workflow versioning	Many long-lived branching workflows
Synchronous API	Caller waits for completion	Simple short operations	Timeout ambiguity for slow effects	Execution reliably fits request deadline

- IC4 stretch — stronger requirement (cross-region acceptance)
 - Cross-region acceptance needs a globally agreed resource home or a synchronous authority.
 - State the added latency and partition unavailability explicitly.
- IC4 stretch — 10× load
 - Partition operations by tenant home cell.
 - Keep one resource's mutations in one authority.
 - Move history and large errors out of hot rows.
- IC4 stretch — cost and blast radius
 - Provider quota cells prevent one failing provider from exhausting all workers.
 - Durable workflows become attractive when timer and compensation complexity dominates.
- IC4 stretch — migration
 - Version step schemas and drain incompatible executions before removing old workers.
- Weakest assumption
 - The provider offers enough identity or inspection to reconcile unknown effects.
 - Mitigation: prove the reconciliation path per provider before onboarding it, and expose `RECONCILING` with a bounded manual-resolution target instead of promising automatic resolution.
- Over-engineering warning

- Do not add distributed transactions to pretend an arbitrary provider joins your SQL commit.

Level signal — IC3 baseline vs IC4 stretch

Phase	IC3 must land	IC4 adds	Separating signal
Scope	Separate acceptance from completion	Negotiate provider uncertainty contract	Names what cannot be guaranteed
Capacity	Size active work using mean duration	Model catch-up and provider quotas	Controls overload after recovery
Architecture	API, outbox, queue, leased worker, and one metadata authority with one trace	States the reconciliation each split creates and its cost	Every component has a mechanism and a price
Correctness	Atomic intent and fenced transitions	Cross-region resource ownership	No stale completion
Operations	Reconcile unknown outcomes	Version long-lived workflows	Recovery survives deployment
Wrap	Recap and the provider-uncertainty tradeoff	Names the weakest assumption before being asked	Identifies the flaw first

Interview pushback and validation

- Q1: Why not keep the HTTP connection open?
 - **IC3 baseline**
 - A lost connection does not reveal whether provisioning happened.
 - **IC4 stretch**
 - Keep synchronous behavior only for operations with a short bounded effect and the same durable identity.
- Q2: Two requests reuse one token with different bodies?
 - **IC3 baseline**
 - Compare canonical hashes and return 409.
 - **IC4 stretch**
 - Version canonicalization so rolling API deployments agree on semantic identity.
- Q3: The worker created the resource and crashed?
 - **IC3 baseline**
 - Reconcile by the persisted provider token or correlation tag before retrying.
 - **IC4 stretch**
 - Quarantine unresolvable ambiguity rather than claim exactly-once creation.
- Q4: Can cancel guarantee no resource exists?
 - **IC3 baseline**
 - No
 - It can stop future work or request compensation.
 - **IC4 stretch**
 - Expose a separate cleanup completion contract for costly irreversible effects.
- Q5 (premise reversal): The provider offers no lookup or deduplication, so why keep an idempotency layer at all?
 - **IC3 baseline**
 - Automatic safe retry cannot be promised after an uncertain response.
 - **IC4 stretch**
 - Negotiate manual reconciliation or accept bounded duplicate effects as an explicit product tradeoff.
- Q6: Why do status reads go backwards?
 - **IC3 baseline**
 - A stale replica or cache violates the promised session ordering.
 - **IC4 stretch**

- Require minVersion or authoritative routing and fail explicitly when that bound cannot be met.
- Q7: Why not make the queue the source of truth and drop the metadata transaction?
 - **IC3 baseline**
 - A queue acknowledgment cannot tell a client whether its operation exists, and a lost reply cannot be deduplicated without a durable token record.
 - **IC4 stretch**
 - Keep the queue as transport only; when metadata write load becomes the limit, partition the store by tenant home cell rather than moving authority into the broker.
- Validation scenarios
 - Race 100 matching tokens and verify one logical operation.
 - Crash before and after every acceptance commit and queue acknowledgment.
 - Pause an old worker, reclaim the lease, and reject its late terminal write.
 - Lose the provider reply and prove no blind duplicate create occurs.
 - Race cancel with provider completion and verify compensation status.
 - Replay completion events and verify one transition per version.
 - Deploy a new worker version mid-flight and verify old step intents complete.

Recall card

- "202 means durable intent, not completed infrastructure."
- "The outbox closes the database-to-queue gap."
- "A stale worker needs a fence at publication."
- "An unknown provider effect needs reconciliation."
- Wrap-up script
 - "The API commits resource intent and an operation before returning 202."
 - "Leased workers execute persisted steps and reconcile uncertain provider effects."
 - "Versioned status and transactional outbox events make recovery observable."
 - "The main limit is what the external provider lets us deduplicate or inspect."

References

- [Ranked bank](#): prompt and provenance.
- [Role framing](#): IC3 baseline, IC4 stretch, and round clock.
- [OCI role framing](#): req decode and the control-plane vocabulary that scores.
- [API/schema companion](#): related contract drills.
- Reference check: 2026-09-10.
 - Official documentation supports the mechanisms cited next to each link.
 - Capacity values and architecture choices remain proposed interview assumptions.
- [OCI work requests](#): asynchronous operation tracking
 - Support varies by API.
- [Transactional outbox](#): atomic state and event intent.